COMP 511

Computer Programming Foundations II

Graphs

Graphs

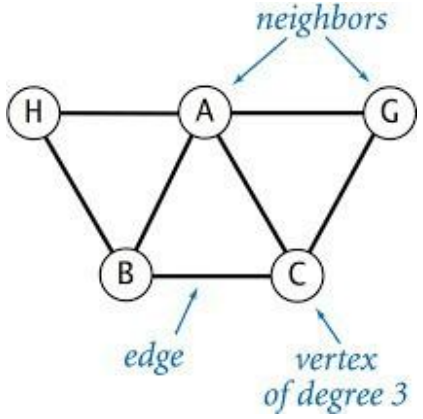
A graph **G** is an *ordered pair* of a set **V** of vertices, and a set **E** of edges.

G = (V,E)

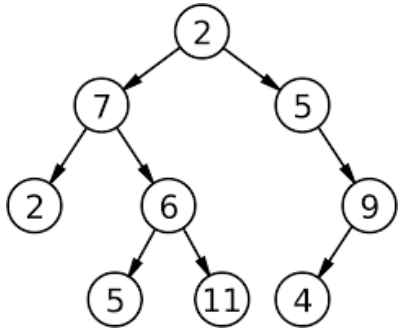
In the image to the right:

V={A,B,C,G,H}

E={ {A,B}, {A,C}, {A,G}, {A,H}, {B,C}, {B,H}, {C,G} }



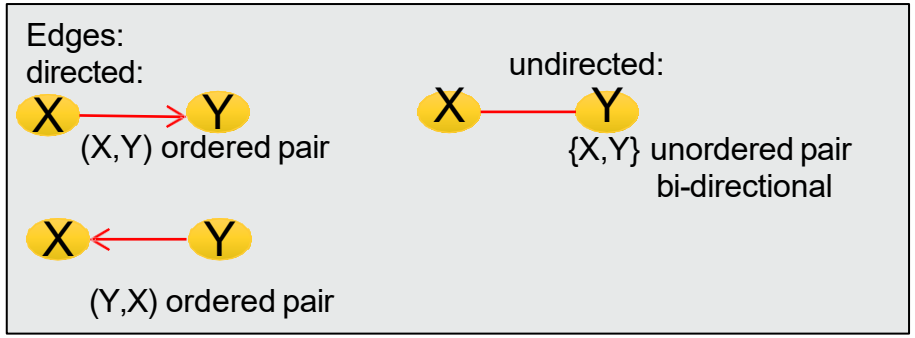
directed graphs



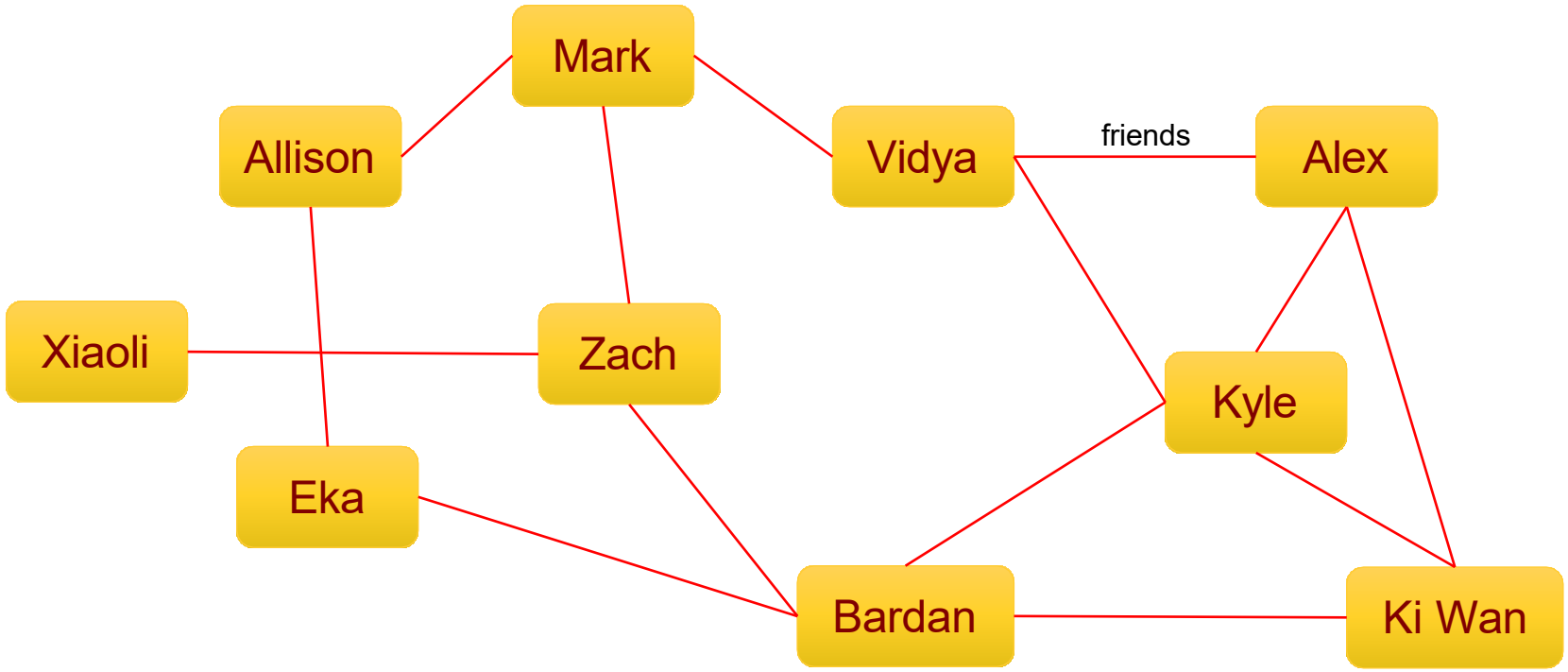
Graphs can be directed, undirected, or both (in some cases)

A **path** is a sequence of edges which connect a sequence of vertices.

A path that does not repeat vertices is called a **simple path**

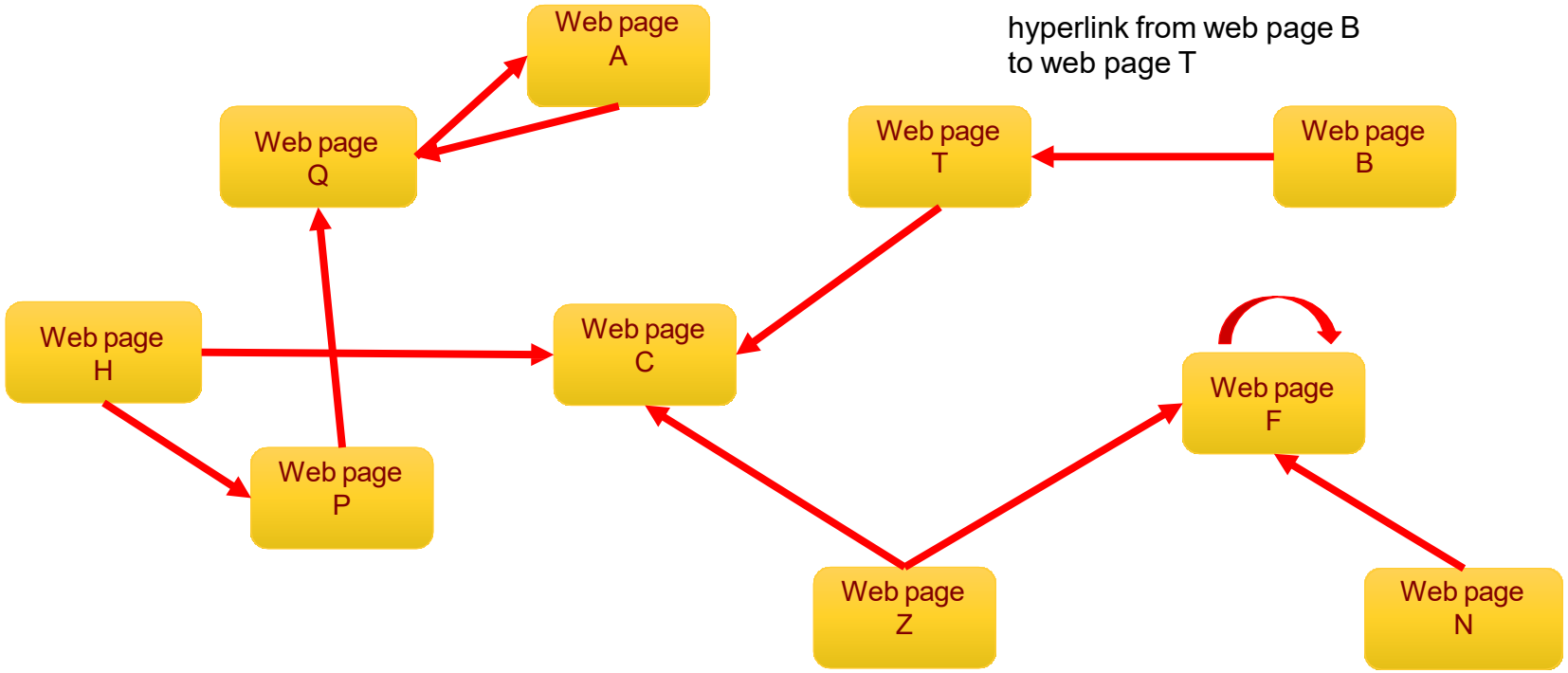


Undirected Graph Example



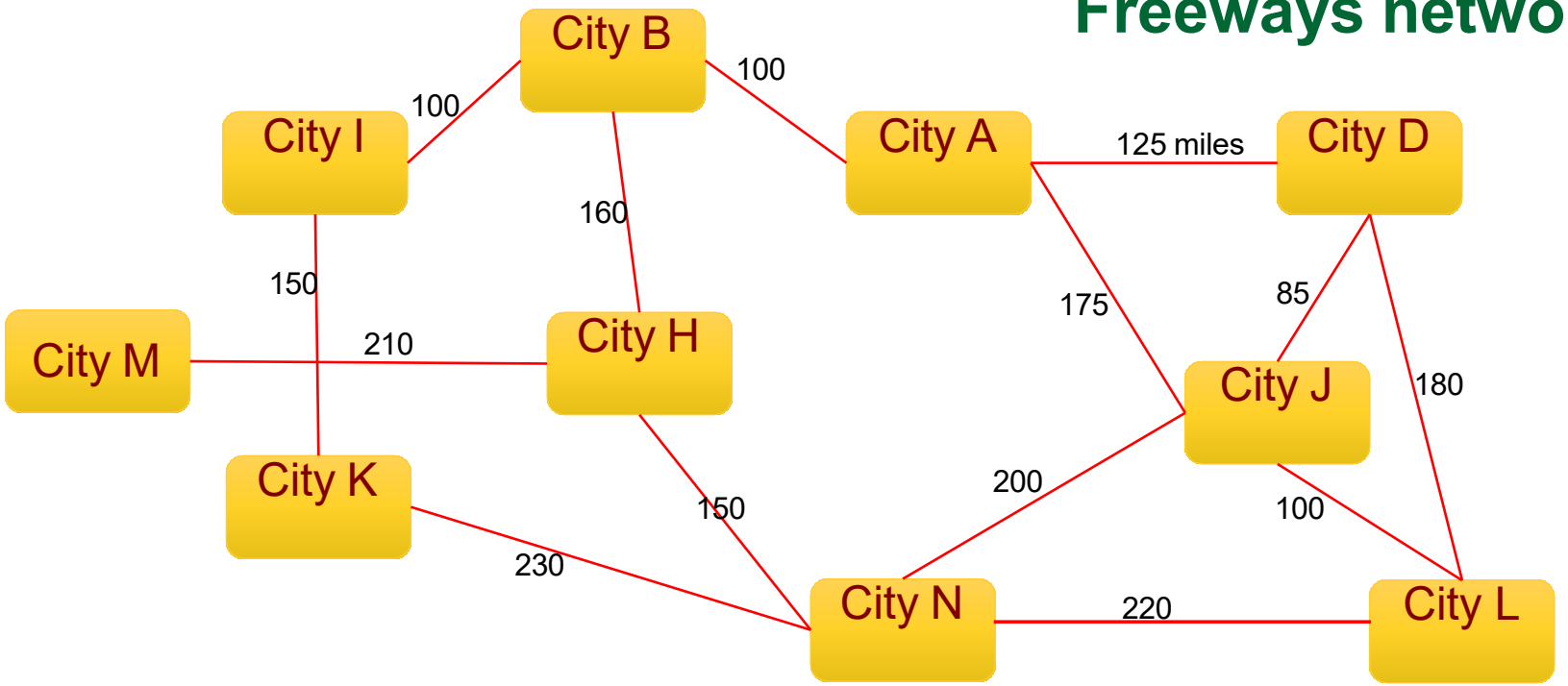
Friends of friends? Can be implemented as paths = 2

Directed Graph Example



Weighted Graphs

Freeways network



Not all freeways are the same of course, for instance their lengths vary.
Let's say we wish to pick the shortest route from City N to City D.
There are numerous options. Here are just a few:
((City N, City J), (City J, City D))
((City N, City L), (City L, City D))
((City N, City J), (City J, City A), (City A, City D))
((City N, City L), (City L, City J), (City J, City D))

Implementation of Undirected/unweighted graph

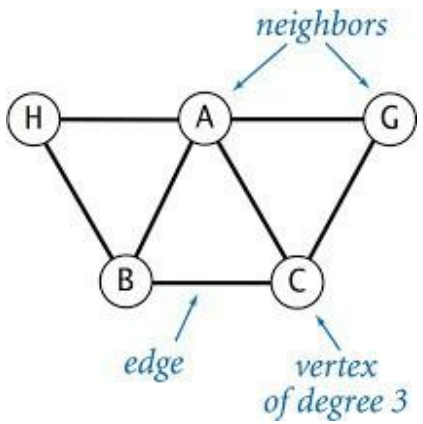
To review, a graph G is an *ordered pair* of a set V of vertices, and a set E of edges.

G = (V,E)

In the image to the right:

V={A,B,C,G,H} and

E={ {A,B}, {A,C}, {A,G}, {A,H}, {B,C}, {B,H}, {C,G} }



Let's create 2 lists:

1. Vertex list
2. Edge list

Vertex List
A
B
C
G
H

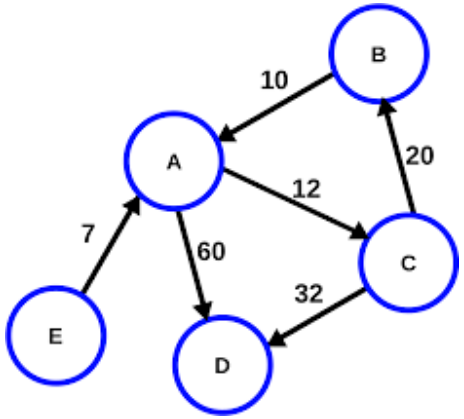
Edge List	
vertex1	vertex2
A	B
A	C
A	G
A	H
B	H
B	C
C	G

```
class UndirectedEdge
{
    string vertex1;
    string vertex2;
};

String vertex_list[MAX_SIZE]
UndirectedEdge edge_list[MAX_SIZE]
```

Note: In this example, since the graph Is *undirected*, class UndirectedEdge stores an *unordered* pair of vertices.

Implementation of Directed/Weighted graph



Vertex List
A
B
C
D
E

Edge List		
startVertex	endVertex	weight
A	C	12
A	D	60
B	A	10
C	B	20
C	D	32
E	A	7

```
Class DirectedEdge
{
    string startVertex;
    string endVertex;
    int weight;
};

String vertex_list[MAX_SIZE]
DirectedEdge edge_list[MAX_SIZE]
```

<u>Note</u>: In this example, since the graph Is *directed*, class DirectedEdge stores an *ordered* pair of vertices.

Space complexity

Cost:

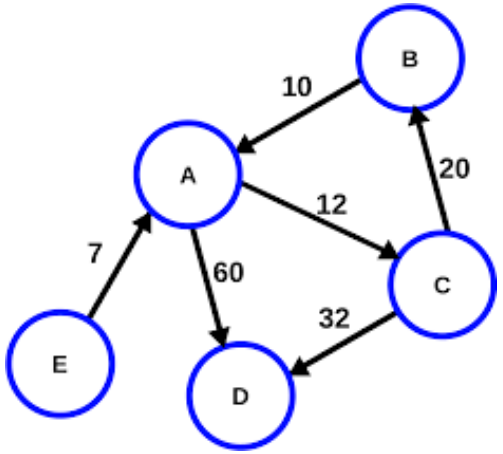
- Memory usage (space complexity)
- Time-cost of operations (time complexity)

O(|v|)

For the Vertex List, space complexity Is proportional to the number of vertices (elements) in set V.

For the Edge List, space complexity may be reduced by using references or indexes into the Vertex list. We have **O(|E|)** for the number of edges in set E, since references and indexes are of constant size.

Total Space: O(|V| + |E|)



Vertex List
A
B
C
D
E

Edge List

startVertex	endVertex	weight
		12
A	D	60
B	A	10
C	B	20
C	D	32
E	A	7

Time complexity

How much time is required to find all nodes adjacent to a given node?

- One way is to sequentially go through the Edge List and determine whether the startVertex (or endVertex if undirected) is our given node. Therefore time complexity is O(|E|).

Another very similar question is how to determine if any specified two nodes are connected. Again, we can sequentially go through the Edge List, and again the time complexity is O(|E|).

- With n nodes, there are n - 1 *directed* edges that can lead from each node to every other node. Therefore, the maximum number of edges is n * (n - 1).

If |V| = n
Then 0 <= |E| <= n * (n-1) if the graph is directed
Or 0 <= |E| <= (n * (n-1))/2 if the graph is undirected.
Therefore in a worst-case scenario, we're essentially working with $O(n^2)$

- Since this number of edges can get very large, it would be preferable to base the time complexity on the number of nodes (vertices) rather than the number of edges, since O(|V|) is much less than $O(n^2)$.

Adjacency Matrix

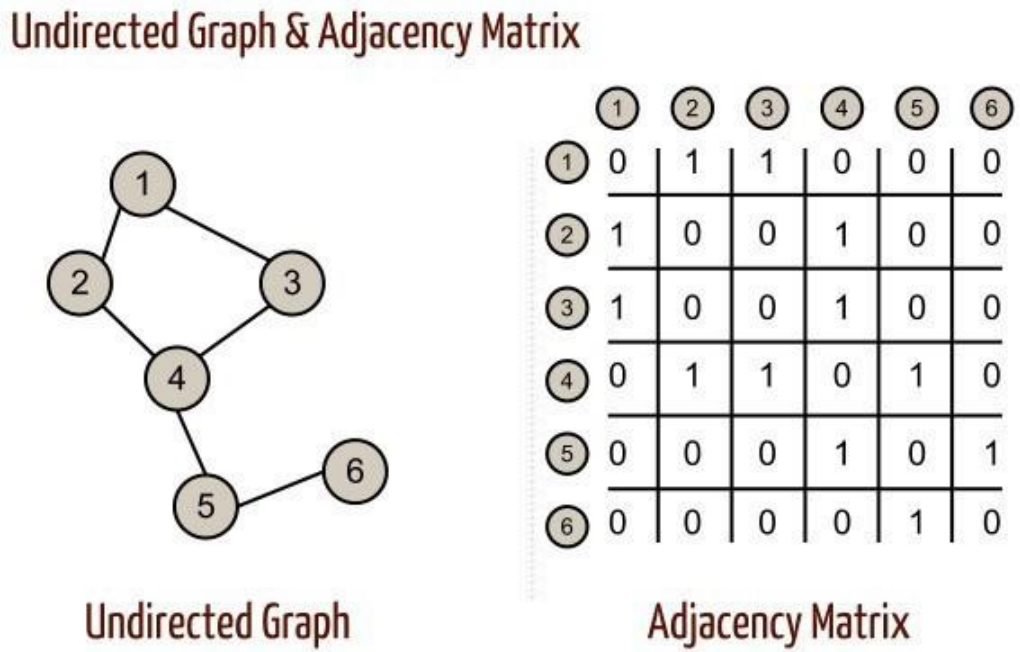
Let's set up a 2-dimensional (n x n) array **A** where n is the number of vertices (nodes) within the graph.

Let i=row, j=column

A_{ij} = **1,** if there exists an edge connecting i to j

0, otherwise

Notice that for an **undirected** graph the adjacency matrix is **symmetrical**.



	1	2	3	4	5	6
1	0	1	1	0	0	0
2	1	0	0	1	0	0
3	1	0	0	1	0	0
4	0	1	1	0	1	0
5	0	0	0	1	0	1
6	0	0	0	0	1	0

Adjacency Matrix

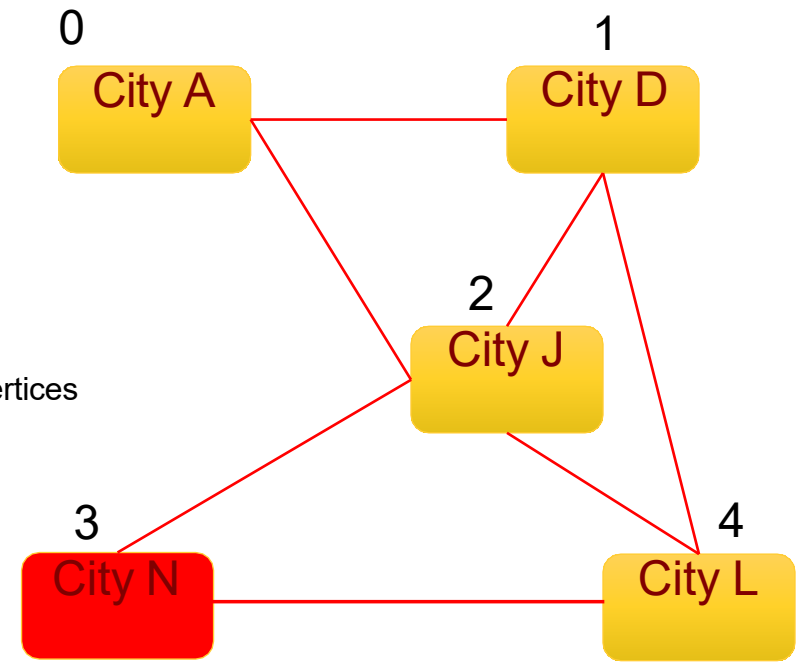
The time cost of finding all nodes adjacent to a given node = **O(v)**

What would be the time cost of determining whether two nodes are connected?

Note: With adjacency matrices we save **time**, but at what cost to **space**? Adjacency matrices are only efficient when the graph is dense rather than sparse.

Index	Vertex List
1	City A
2	City D
3	City J
4	City N
5	City L

O(v) v = # of vertices



Adjacency Matrix

	0	1	2	3	4
0	0	1	1	0	0
1	1	0	1	0	1
2	1	1	0	1	1
3	0	0	1	0	1
4	0	1	1	1	0

O(v)

Graph Representation

V = # of vertices

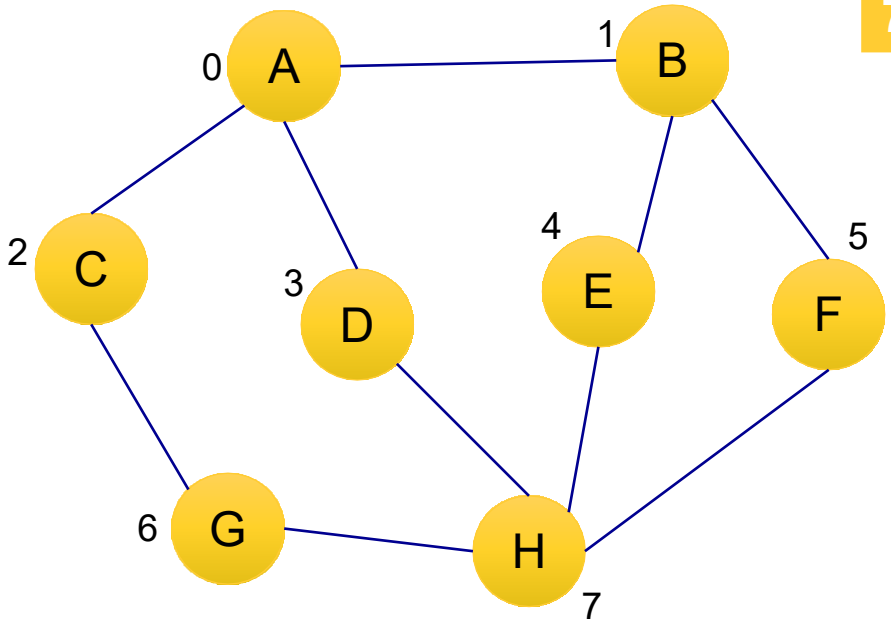
Recall: With a static adjacency matrix, finding adjacent nodes has a time-cost of O(v) while finding if two nodes are connected has a time-cost of O(1) assuming we know their indices within the vertex list.

Vertex List

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

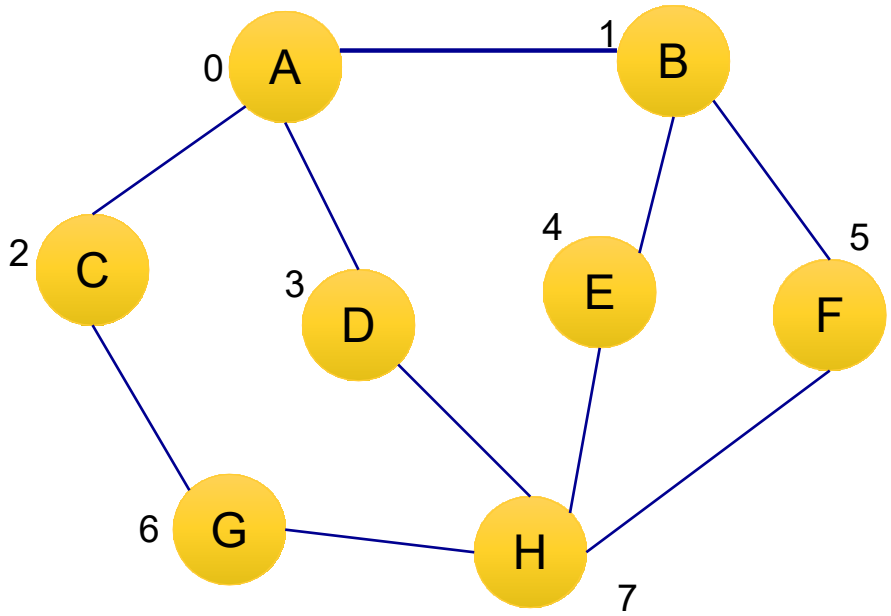
Adjacency Matrix

	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0
1	1	0	0	0	1	1	0	0
2	1	0	0	0	0	0	1	0
3	1	0	0	0	0	0	0	1
4	0	1	0	0	0	0	0	1
5	0	1	0	0	0	0	0	1
6	0	0	1	0	0	0	0	1
7	0	0	0	1	1	1	1	0



Graph Representation

Alternatively, how about a linked list?



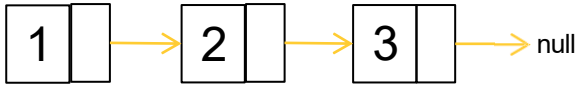
Vertex List

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

Adjacency Matrix

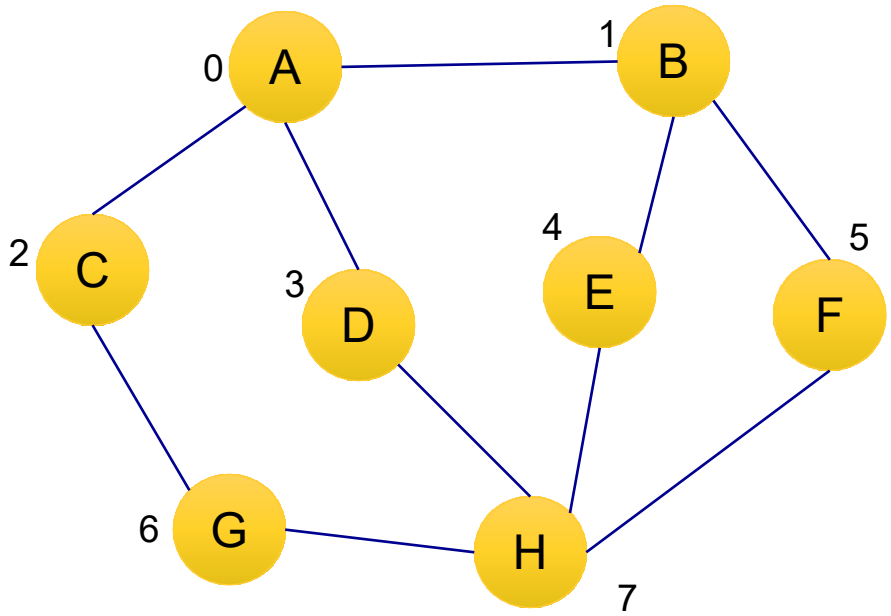
	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0

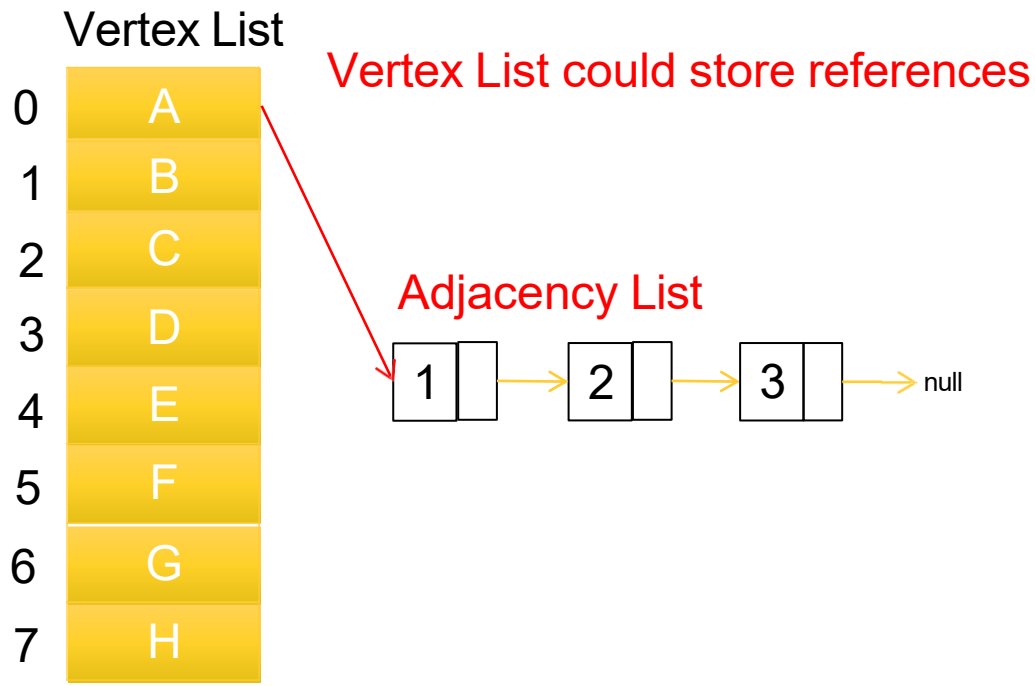
Adjacency List



Graph Representation

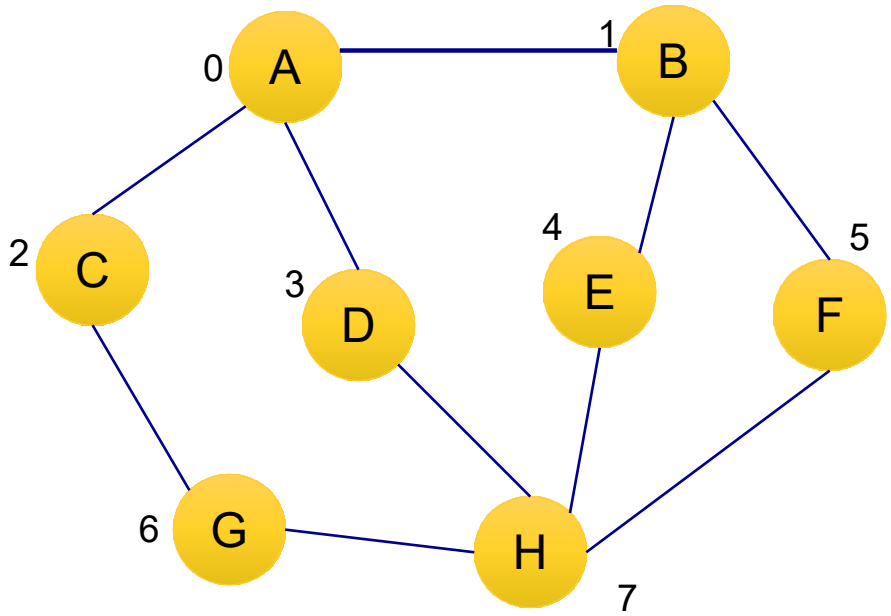
Alternatively, how about a linked list?





Graph Representation

Or, (perhaps) better yet, a bst!

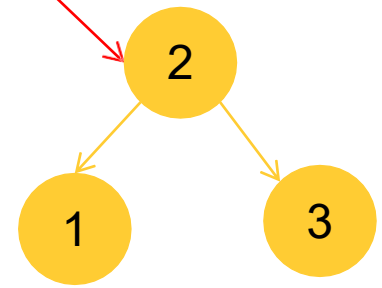


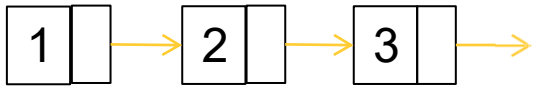
Vertex List

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

Adjacency Matrix

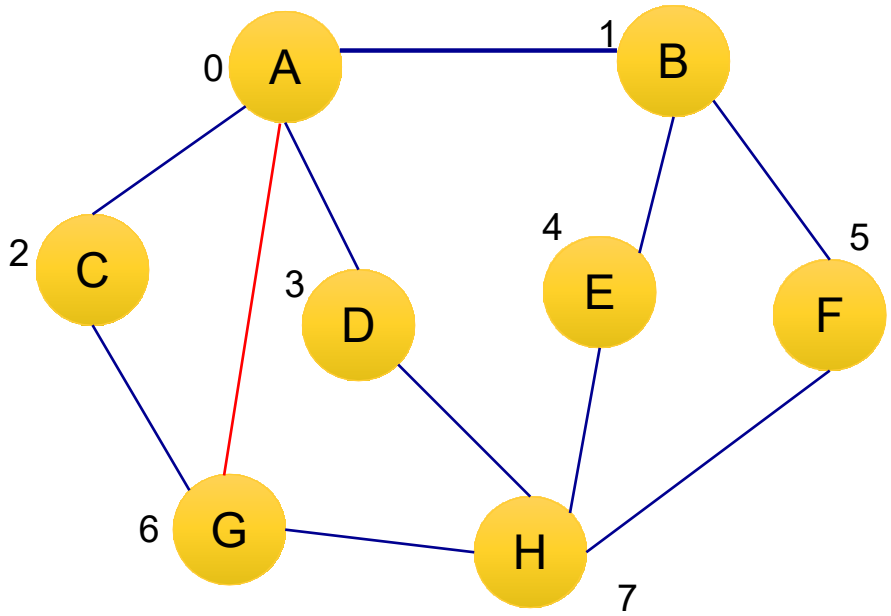
	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0





Graph Representation

What if we add a new edge?

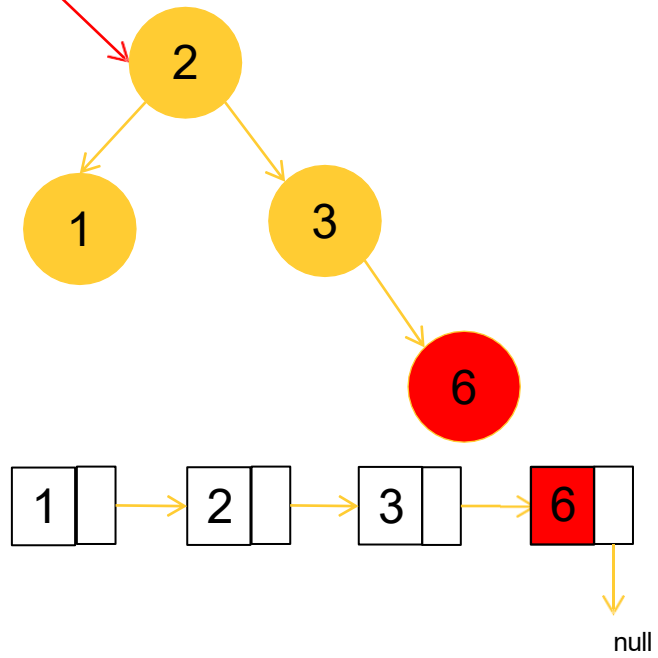


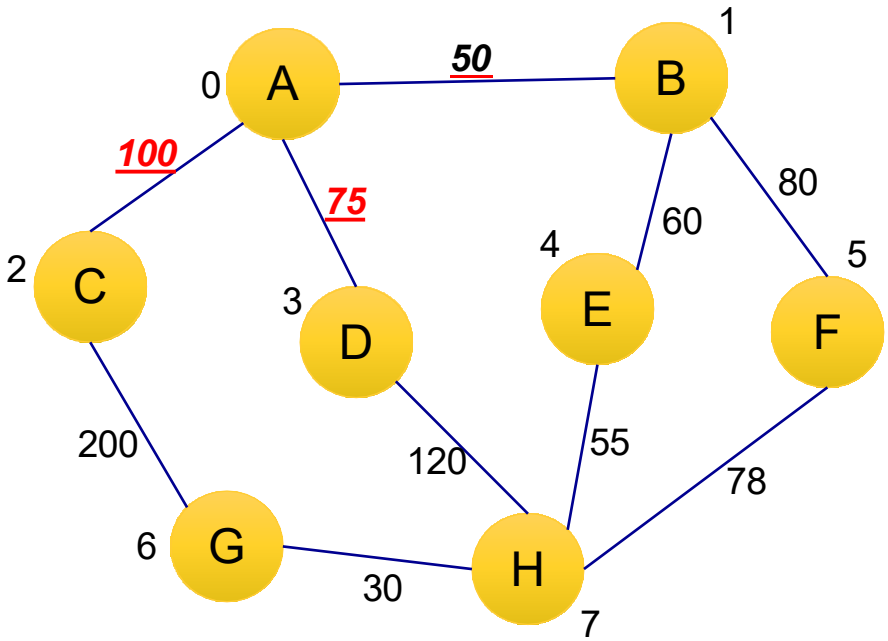
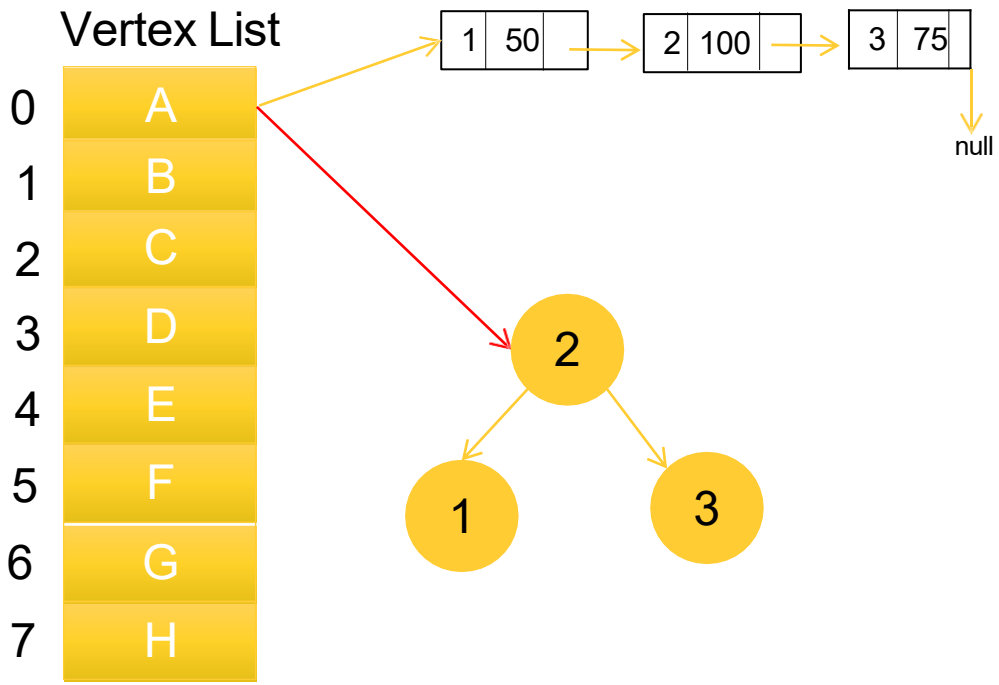
Vertex List

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

Adjacency Matrix

	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	1	0



Graph Representation

How about for a weighted graph?